



International Islamic University Chittagong

# IIUC\_StrawHats

Afrin Hasan Safin, Jillur Rahman, Md. Arif

2025-11-20

Basics and Misc (1)

1.1 Basics.

Prefix Sum 2D.cpp

**Description:** Computes a 2D prefix-sum array in  $O(nm)$ . For a matrix  $a$ , the prefix matrix  $pref$  satisfies:

pref[i][j] = \sum\_{x=1}^i \sum\_{y=1}^j a[x][y].

A sub-rectangle sum  $(x_1, y_1)$  to  $(x_2, y_2)$  is:

pref[x2][y2]-pref[x1-1][y2]-pref[x2][y1-1]+pref[x1-1][y1-1]

**Usage:** Build pref[][] once, then answer each query in  $O(1)$ .

**Time:**  $O(nm)$  build,  $O(1)$  per query.

```
const int N = 1005;
int a[N][N], pref[N][N]; // ykw
int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int n, m; cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            pref[i][j] = pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1] + a[i][j];
        }
    }
    int q; cin >> q;
    while (q--) {
        int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
        int ans = 0;
        // for (int i = x1; i <= x2; i++) {
        //     for (int j = y1; j <= y2; j++) {
        //         ans += a[i][j];
        //     }
        // }
        ans = pref[x2][y2] - pref[x1 - 1][y2] - pref[x2][y1 - 1] + pref[x1 - 1][y1 - 1];
        cout << ans << '\n';
    }
    return 0;
}
```

Bitmask and bitwise.cpp

**Description:** Utilities for bitmask enumeration and bit operations. Enumerating all subsets of  $k$  elements takes  $2^k$  iterations. Bit operations use 0-based indexing:

**Usage:** for (mask = 0; mask < (1 << k); mask++) iterate subsets.

**Time:**  $O(2^k)$  for enumeration,  $O(1)$  for each bit op.

```
//getting all possible subset of k elements(k elements has 2^k subsets)
void bitmask(int k) {
    for (int j = 0; j < (1 << k); j++) {
        for (int i = 0; i < k; i++) {
            if (j & (1 << i)) {
```

```
                // * take the i-th element
            }
        }
    }
    //bitwise basics 0-based index
#define cnt_bits(x) __builtin_popcountll(x)
#define clz(x) __builtin_clzll(x)
#define ctz(x) __builtin_ctzll(x)

ll set0(ll n, ll k) {
    n = (n | (1LL << k)) ^ (1LL << k);
    return n;
}
ll set1(ll n, ll k) {
    n = (n | (1LL << k));
    return n;
}
ll tog1(ll n, ll k) {
    return (n ^ (1LL << k));
}
bool is_set(ll n, ll k) {
    return (n & (1LL << k));
}

bool is_power_of_2(ll n) {
    return (n > 0 && !(n & (n - 1)));
}
```

compare.cpp

**Description:** Two comparison helpers. Pair comparator: sorts by first element ascending; if equal, sorts by second element descending. Formally:  $(A < B)$  iff  $(A.first < B.first)$  or  $(A.first = B.first \wedge A.second > B.second)$ . Double comparator: compares two floating-point numbers with epsilon. Returns 0 if  $|a - b| \leq \epsilon$ , otherwise  $-1$  if  $a < b$ , and 1 if  $a > b$ .

**Usage:** sort(v.begin(), v.end(), cmp); if (double comp(x, y) == 0) { equal }

**Time:**  $O(1)$

```
bool cmp(pair<int, int> A, pair<int, int> B) {
    // pair compare
    if (A.first == B.first)
        return (A.second > B.second);
    //! if A.second is Larger than A will come before B
    return (A.first < B.first);
    //! if A.first is smaller than A will come before B
}

int double_comp(ld a, ld b) {
    //compare between 2 double
    if (fabs(a - b) <= eps)
        return 0;
    return a < b ? -1 : 1;
    //! dbl cmp return 0 when a==b, -1 if a<b, 1 if a>b
}
```

Direction Array.cpp

**Description:** Direction vectors for grid and knight moves. - 4/8 directions  $(dx[i], dy[i])$ ,  $i = 0..7$ :  $(+1, 0) = \text{Down}$ ,  $(-1, 0) = \text{Up}$ ,  $(0, +1) = \text{Right}$ ,  $(0, -1) = \text{Left}$ ,  $(+1, +1) = \text{Down-Right}$ ,  $(+1, -1) = \text{Down-Left}$ ,  $(-1, +1) = \text{Up-Right}$ ,  $(-1, -1) = \text{Up-Left}$  - Knight moves  $(kdx[i], kdy[i])$ ,  $i = 0..7$

**Time:**  $O(1)$

```
7b6242, 7 lines
// (dx, dy for grid-based problems) 4 directions and 4 diagonal moves
const ll dx[] = {1, -1, 0, 0, 1, 1, -1, -1};
const ll dy[] = {0, 0, 1, -1, 1, -1, 1, -1};

// Knight's Move
const ll kdx[] = {-2, -2, -1, -1, 1, 1, 2, 2};
const ll kdy[] = {-1, 1, -2, 2, -2, 2, -1, 1};
```

Maximum subarray sum.cpp

**Memory:**  $O(N)$

**Time:**  $O(N)$

```
e61165, 9 lines
ll kadane_algo(const vector<ll> &v, ll n) {
    ll temp = 0, res = 0;
    for (ll i = 0; i < n; i++) {
        temp += v[i];
        if (temp < 0) temp = 0;
        res = max(res, temp);
    }
    return res;
}
```

1.2 Misc.

Dates.h

**Description:** Provides functions to convert between Gregorian calendar dates and Julian Day numbers. - 'dateToInt(y, m, d)': Converts a date (year, month, day) to its corresponding Julian Day number. - 'intToDate(jd, y, m, d)': Converts a Julian Day number back to a Gregorian date. - 'intToDay(jd)': Returns the day of the week (0 = monday, 5 = Saturday) from a Julian Day number.

**Usage:** int jd = dateToInt(2025, 11, 14);

int y, m, d; intToDate(jd, y, m, d); int weekday = intToDay(jd);

**Memory:**  $O(1)$  for all functions

**Time:**  $O(1)$  for all functions

```
892297, 20 lines
int intToDay(int jd) { return jd % 7; }
int dateToInt(int y, int m, int d) {
    return 1461 * (y + 4800 + (m - 14) / 12) / 4 +
        367 * (m - 2 - (m - 14) / 12 * 12) / 12 -
        3 * ((y + 4900 + (m - 14) / 12) / 100) / 4 +
        d - 32075;
}
void intToDate(int jd, int &y, int &m, int &d) {
    int x, n, i, j;
    x = jd + 68569;
    n = 4 * x / 146097;
    x -= (146097 * n + 3) / 4;
    i = (4000 * (x + 1)) / 1461001;
    x -= 1461 * i / 4 - 31;
    j = 80 * x / 2447;
```

```
d = x - 2447 * j / 80;
x = j / 11;
m = j + 2 - 12 * x;
y = 100 * (n - 49) + i + x;
}
```

Random.h

**Description:** gen\_random(l, r); random number deil to r er modde, shuffle() vector ke randomly suffle kore.

```
2ef017, 6 lines
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
inline ll gen_random(ll l, ll r) {
    return uniform_int_distribution<ll>(l, r)(rng);
}
```

shuffle(all(v), rng);

LIS.h

**Description:** Compute indices for the longest increasing subsequence.

**Time:**  $O(N \log N)$

```
2932a0, 17 lines
template<class I> vi lis(const vector<I>& S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i, 0, sz(S)) {
        // change 0 -> i for longest non-decreasing subsequence
        auto it = lower_bound(all(res), p(S[i], 0));
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it-1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
```